

□ Project Documentation: AI-Powered Support Assistant (A-Z)

1. Project Introduction

The **AI Support Assistant** is a full-stack web application designed to simulate a professional customer support experience. It uses Large Language Models (LLM) to provide grounded, accurate responses based on internal product documentation, ensuring that the AI never "hallucinates" facts outside the company's policy.

2. Technical Stack (A-Z)

- **Frontend (UI/UX):**
 - **React.js:** A component-based library for building the user interface.
 - **Tailwind CSS:** A utility-first CSS framework used for the premium dark-themed aesthetic.
 - **Lucide React:** For professional, high-quality iconography (User icons, Message bubbles, etc.).
 - **Vite:** The build tool ensuring blazing fast development and optimized production bundles.
 - **Backend (Logic/API):**
 - **Node.js & Express:** The engine that handles API requests, security, and LLM communication.
 - **Dotenv:** For secure management of API keys and server ports.
 - **Express-Rate-Limit:** To prevent malicious spamming of the API.
 - **Data & AI:**
 - **SQLite3:** A lightweight, file-based SQL database chosen for its speed and zero-configuration requirement.
 - **Google Gemini 2.5 Flash:** A state-of-the-art LLM that balances speed and high intelligence for customer support tasks.
-

3. Key Features & Functionality

A. Persistent Conversation History

Every chat is stored in an SQLite database. When a user creates a "New Chat," a unique UUID is generated and stored. Even if you close the browser, your history is retrieved from the database next time you visit.

B. Smart Sidebar Titles

Instead of showing random ID numbers, the sidebar automatically analyzes your messages. It skips generic greetings like "Hi" or "Hello" and finds the first meaningful question to use as the title of that chat.

C. Persistent Token Usage

Unlike standard demos, this project tracks the actual cost (tokens) of the AI's response and saves it in the database. Each session shows its own specific total token count.

D. Strict Grounding Rules

The AI follows a "Knowledge ONLY" policy. It is strictly forbidden from answering general questions (like math or history) unless the answer is present in the `docs.json` file.

4. Database Architecture

The project uses two interlinked tables:

Table: `sessions`

Field	Type	Role
<code>id</code>	TEXT	Primary Key (UUID)
<code>created_at</code>	DATETIME	Tracks when the session began
<code>updated_at</code>	DATETIME	Updates whenever a message is sent

Table: `messages`

Field	Type	Role
<code>id</code>	INTEGER	Primary Key (Auto-increment)
<code>session_id</code>	TEXT	Foreign Key (Links to <code>sessions</code>)
<code>role</code>	TEXT	Either 'user' or 'assistant'
<code>content</code>	TEXT	The actual text message
<code>tokens_used</code>	INTEGER	The tokens used for that specific AI response

5. AI Reasoning & Prompt Engineering

The core logic resides in `Backend/routes/chat.js`. We construct a multi-part prompt for Gemini:

- Context:** We inject the `docs.json` as the "Knowledge Base."
 - Rules:** We provide strict instructions (e.g., "If answer not found, say Sorry...").
 - History:** we inject the **last 5 pairs** (10 messages total) from SQLite so the AI remembers what was said 2 minutes ago.
 - User Input:** The current question.
-

6. How to Run (Step-by-Step)

1. **API KEY:** Place your Gemini API key in `Backend/.env`.
 2. **BACKEND:** Run `cd Backend && npm install && node server.js`.
 3. **FRONTEND:** Run `cd Frontend && npm install && npm run dev`.
 4. **ACCESS:** Visit `http://localhost:8080`.
-

7. Compliance with Requirements

- ☒ Full-Stack (React/Node)
- ☒ SQLite DB Implementation
- ☒ Document-Bases Answers (docs.json)
- ☒ Session Stability (localStorage + UUID)
- ☒ Persistent Token Tracking
- ☒ Professional Error Handling